Kick Start Server

Task 1: Install Kickstart tools and verify

Step 1: Become root

su -

Step 2: Install the package

dnf install pykickstart httpd curl -y

- pykickstart — provides Kickstart validation tools.
- httpd — Apache web server for hosting the file.
- curl — tests whether the file is accessible over HTTP.

Step 3: Verify the installation

rpm -q pykickstart

Check the Kickstart validator:

command -v ksvalidator

You should see a path similar to:

/usr/bin/ksvalidator

The ksvalidator command checks the syntax and supported options in your Kickstart file.

Task 2: Create the ks.cfg configuration file

Step 1: Create the file

mkdir -p /var/www/html

nano /var/www/html/ks.cfg

Enter this basic Kickstart configuration for Rocky Linux 9 or RHEL 9:

```
# Basic Kickstart Configuration

text
reboot

lang en_US.UTF-8
keyboard --xlayouts='us'
timezone Asia/Kolkata --utc
```

```
network --bootproto=dhcp --device=link --activate

rootpw --plaintext Linux@12345
user --name=student --groups=wheel --password=Student@12345 --plaintext

firewall --enabled --service=ssh
selinux --enforcing

bootloader --location=mbr
zerombr
clearpart --all --initlabel
autopart

%packages
@^minimal-environment
%end
```

Save with Ctrl+O, press Enter, and exit using Ctrl+X.

Important: The example passwords are for a disposable lab only. Change them before using the file. --plaintext means the passwords are stored in readable form in ks.cfg; for a real deployment, use hashed passwords with --iscrypted.

Step 2: Understand the main settings

Setting	Purpose
lang	Sets the system language.
timezone	Sets the timezone to India Standard Time.
rootpw	Sets the root account password.
user	Creates the student user.
network	Enables DHCP networking during installation.
clearpart --all	Removes existing partitions on the installation target disk.
autopart	Automatically creates the required partitions.

%packages Specifies which software groups to install.

Warning: This configuration erases partitions on the disk selected by the installer. Use it only on a test VM with no important data. For production or multi-disk systems, explicitly select and verify the target disk.

Step 3: Validate the Kickstart file

Run:

ksvalidator -v RHEL9 /var/www/html/ks.cfg

If the command returns no output, the file passed the syntax check.

This validates the configuration syntax, but it does not guarantee that every package is available or that installation will succeed. A full installation also needs a compatible Rocky/RHEL installation source, such as an ISO or installation repository.

Task 3: Host ks.cfg using Apache

Step 1: Start Apache

systemctl enable --now httpd

systemctl status httpd

Confirm the service is active (running).

Step 2: Set file permissions

chmod 644 /var/www/html/ks.cfg

restorecon -v /var/www/html/ks.cfg

These commands allow the web server to read the file and restore its standard SELinux file context.

Step 3: Allow HTTP through the firewall

firewall-cmd --permanent --add-service=http

firewall-cmd --reload

Step 4: Find your Linux server IP

hostname -I

For example, suppose the output includes 192.168.1.128. Use your actual IP address instead.

Step 5: Test locally

curl http://localhost/ks.cfg

The Kickstart configuration should appear in the terminal.

Step 6: Test from another machine

On another Linux machine, run:

curl http://SERVER_IP/ks.cfg

Replace SERVER_IP with your server's IP address.

Alternatively, open this URL in a browser:

http://SERVER_IP/ks.cfg

If the configuration file appears, the HTTP hosting task is complete.

Task 4: Install GNOME automatically and create welcome.txt

Now we will update the Kickstart file to install the GNOME desktop environment and create a welcome message after installation.

Step 1: Edit the file

nano /var/www/html/ks.cfg

Replace the existing configuration with this updated version:

```
# Kickstart Configuration with GNOME

text
reboot

lang en_US.UTF-8
keyboard --xlayouts='us'
timezone Asia/Kolkata --utc

network --bootproto=dhcp --device=link --activate

rootpw --plaintext Linux@12345
user --name=student --groups=wheel --password=Student@12345 --plaintext

firewall --enabled --service=ssh
selinux --enforcing

bootloader --location=mbr
zerombr
clearpart --all --initlabel
```

autopart

%packages

@^graphical-server-environment

%end

%post --log=/root/ks-post.log

mkdir -p /home/student

echo "Welcome to your automated Linux setup!" > /home/student/welcome.txt

chown student:student /home/student/welcome.txt

chmod 644 /home/student/welcome.txt

%end

Save and exit.

What changed?

1. GNOME desktop environment

%packages

@^graphical-server-environment

%end

This requests the graphical server environment, which includes a GNOME desktop on supported RHEL/Rocky 9 installation sources. The required package group must exist in your installation repository.

2. Post-installation script

%post --log=/root/ks-post.log

mkdir -p /home/student

echo "Welcome to your automated Linux setup!" > /home/student/welcome.txt

chown student:student /home/student/welcome.txt

chmod 644 /home/student/welcome.txt

%end

The %post section runs commands after installation. It creates the file, writes the message, assigns ownership to student, and sets read permissions.

Step 2: Validate the updated configuration

ksvalidator -v RHEL9 /var/www/html/ks.cfg

If there is no output, the syntax check passed.

Step 3: Restart Apache and verify the updated file

systemctl restart httpd

curl http://localhost/ks.cfg

Confirm that the GNOME package group and %post section appear in the output.

Step 4: Verify after an automated installation

When you install a separate test VM using this Kickstart file, log in as student and run:

cat /home/student/welcome.txt

Expected output:

Welcome to your automated Linux setup!

To verify the desktop environment, run:

systemctl get-default

For a graphical boot target, the output should be:

graphical.target

If necessary, set the default target inside the newly installed VM:

sudo systemctl set-default graphical.target